

Product Requirements Document

React Calculator Web App - Technical Specification for AI Code Generation

1. Project Objective

Create a simple, functional calculator web application using React. The app should perform basic arithmetic operations and feature a clean, user-friendly interface.

2. Tech Stack

- **Framework:** React (using Functional Components and Hooks)
- **Styling:** Standard CSS (using CSS Grid for the layout) or Tailwind CSS
- **Build Tool:** Vite or Create React App

3. Functional Requirements

The calculator must support the following features:

- **Number Input:** Buttons for digits `0-9` and a decimal point `.`.
- **Basic Operations:** Addition (`+`), Subtraction (`-`), Multiplication (`*`), and Division (`/`).
- **Calculation:** An equals button (`=`) to evaluate the expression.
- **Clear Functions:** An All Clear button (`AC`) to reset the entire state, and a Delete button (`DEL`) to remove the last entered character.
- **Error Handling:** Prevent multiple consecutive decimals (e.g., `1..2`) and handle division by zero gracefully (display "Error").

4. Component Architecture

Build the application using a modular component structure:

- `App.jsx` : The root component that renders the calculator layout.
- `Calculator.jsx` : The main wrapper managing the logic and state.
- `Display.jsx` : A component to show the previous operand, current operator, and current operand.
- `Button.jsx` : A reusable button component accepting props for its label and an `onClick` handler.

5. State Management Logic

Use React's `useState` or `useReducer` hook inside the `Calculator` component to manage:

- `currentOperand`: The number currently being typed.
- `previousOperand`: The number typed before an operator was selected.
- `operation`: The mathematical operator currently selected (e.g., `+`, `-`).

6. UI/UX Layout Guidelines

The layout should resemble a standard physical calculator.

- **Container:** Centered on the screen with a subtle shadow and rounded corners.
- **Screen/Display:** Spans the top of the calculator container, right-aligned text.
- **Keypad Grid:** Use a 4-column CSS Grid.

Grid Layout Details:

- Row 1: `AC` (spans 2 columns), `DEL`, `/`
- Row 2: `1`, `2`, `3`, `*`
- Row 3: `4`, `5`, `6`, `+`
- Row 4: `7`, `8`, `9`, `-`
- Row 5: `.`, `0`, `=` (spans 2 columns)

7. Execution Instructions for the AI

1. Initialize the project structure.
2. Create the necessary component files.
3. Implement the state management and calculation logic.
4. Apply CSS Grid styling to match the layout guidelines.
5. Output the full, runnable code.